IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (Deemed to be University)

Yenepoya Institute of Arts Science Commerce and Management

Project-High Level Design

On

Multi Cloud Hybrid Strategy for Smart City

Student Name: Muhammed Abdul Gafoor

Industry Mentor:Sumit kumar shukla

Guided by:Deepthi k

Table of Contents

1. Introduction 3
   1.1. Scope of the document 4
   1.2. Intended Audience 4
   1.3. System overview 4
2. System Design 5
   2.1. Application Design 5
   2.2. Process Flow 6
   2.3. Information Flow 7
   2.4. Components Design 9
   2.5. Key Design Considerations 10
   2.6. API Catalogue 11
3. Data Design 12
   3.1. Data Model 12
   3.2. Data Access Mechanism 12
   3.3. Data Retention Policies 13
   3.4. Data Migration 13
4. Interfaces 14
5. State and Session Management 14
6. Caching 14
7. Non-Functional Requirements 14
   7.1. Security Aspects 15
   7.2. Performance Aspects 15
8. References 15

1. Introduction

The *Multi-Cloud Hybrid Strategy for Smart City* system is designed to provide an efficient, scalable, and fault-tolerant platform for managing citizen complaints and service requests. In modern smart city environments, handling large volumes of data while ensuring availability and reliability is critical. This system addresses these challenges by integrating a hybrid cloud approach that combines local storage with cloud-based backup solutions.

The system enables seamless interaction between users, authorities, and administrators while ensuring that data is securely stored, regularly backed up, and recoverable in case of failures. The hybrid model improves reliability by storing data in multiple locations, reducing the risk of data loss and downtime.

1.1 Scope of the Document

This document presents the High-Level Design (HLD) of the system. It focuses on the structural and architectural aspects rather than implementation details.

The scope includes:

- Overall system architecture and design approach
- Description of major modules and their responsibilities
- Data flow and process flow across the system
- Backup, recovery, and failover mechanisms
- Integration with external services such as cloud storage

This document does not include low-level coding details, algorithms, or implementation-specific logic.

1.2 Intended Audience

This document is intended for multiple stakeholders involved in the development and evaluation of the system:

- **Developers** – to understand system architecture and module interaction
- **System Architects** – to review design decisions and scalability
- **Project Evaluators** – to assess design quality and completeness
- **Academic Reviewers** – for grading and validation of project design

---

1.3 System Overview

The system is a web-based application that allows users to submit complaints related to smart city services such as infrastructure issues, public services, or maintenance problems.

Key functionalities include:

- Users can register and log in to the system
- Users can submit and track complaints
- Administrators can assign complaints to authorities
- Authorities can update complaint status and resolution progress
- Users can provide feedback after resolution

Additional system capabilities:

- Automatic database and media backup
- Storage of backups in both local and cloud environments
- Failover mechanisms for recovery during system failures
- Secure authentication and role-based access control

---

2. System Design

2.1 Application Design

The system follows a **multi-layered architecture** to ensure modularity and scalability.

Presentation Layer

This layer represents the user interface of the system. It is implemented using Django templates (HTML, CSS, and JavaScript). It provides forms and dashboards for users, admins, and authorities.

Responsibilities:

- Accept user input
- Display data dynamically
- Provide a user-friendly interface

Application Layer

This layer contains the core business logic of the system implemented using Django views and controllers.

Responsibilities:

- Process user requests
- Apply validation and business rules
- Coordinate between UI and database
- Trigger backup operations

Data Layer

The data layer manages persistent storage using a MySQL database.

Responsibilities:

- Store structured data such as users, complaints, assignments
- Maintain relationships between entities
- Ensure data integrity and consistency

Backup Layer

This is a critical layer responsible for ensuring data safety.

Responsibilities:

- Perform regular backups of database and media
- Store backups locally and on cloud platforms
- Enable data restoration during failures

2.2 Process Flow

Complaint Management Process

1. User logs into the system
2. User submits a complaint with necessary details
3. Complaint is stored in the database
4. Admin reviews and assigns the complaint to an authority
5. Authority updates status (in-progress/resolved)
6. User receives updates and provides feedback

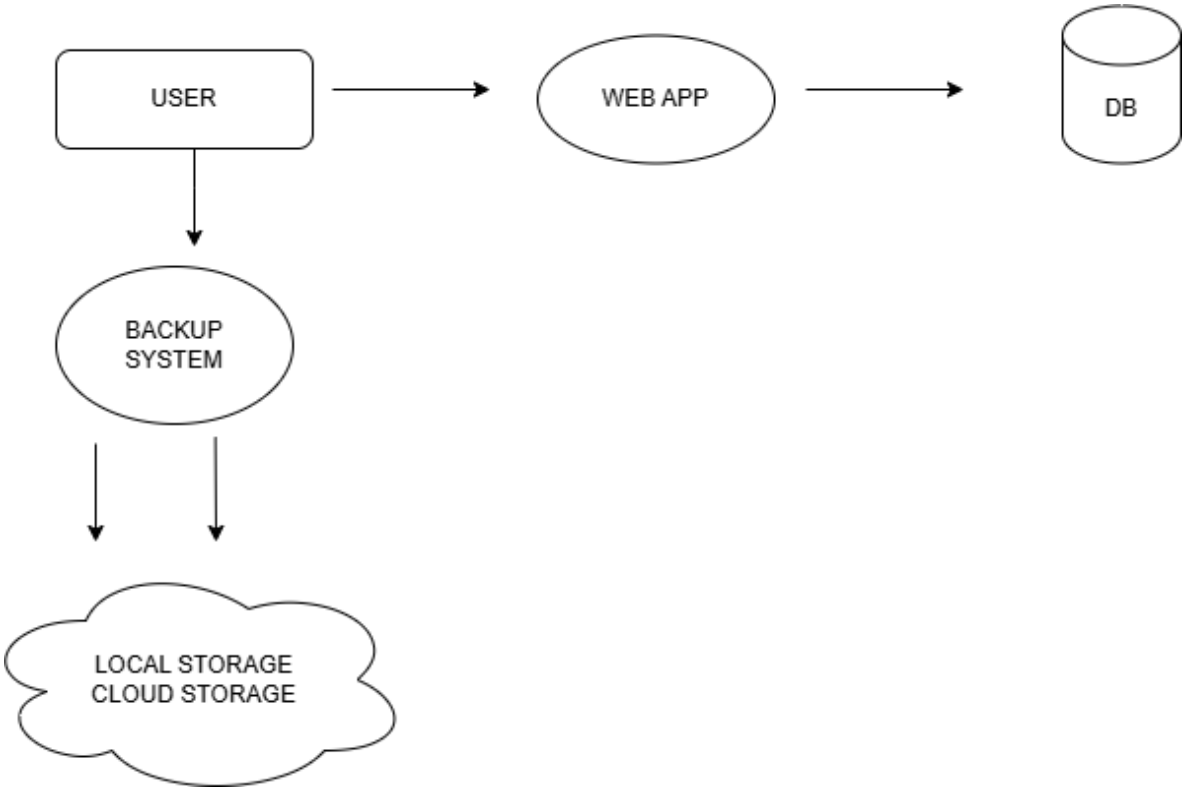
Backup Process Flow

1. Backup is triggered via scheduler (APScheduler) or Django signals
2. Database dump is generated
3. Media files are compressed into archive format
4. Backup is stored in local storage
5. Backup is uploaded to cloud storage (Google Drive)
6. Old backups are deleted based on retention policy

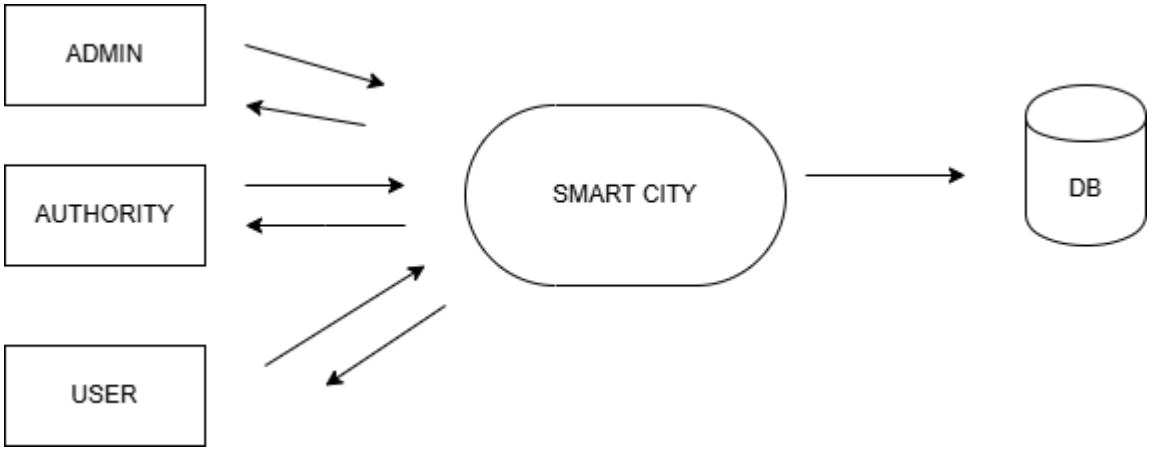
Restore Process Flow

1. System checks for available local backup
2. If unavailable, retrieves backup from cloud
3. Database and media files are restored
4. System resumes normal operation

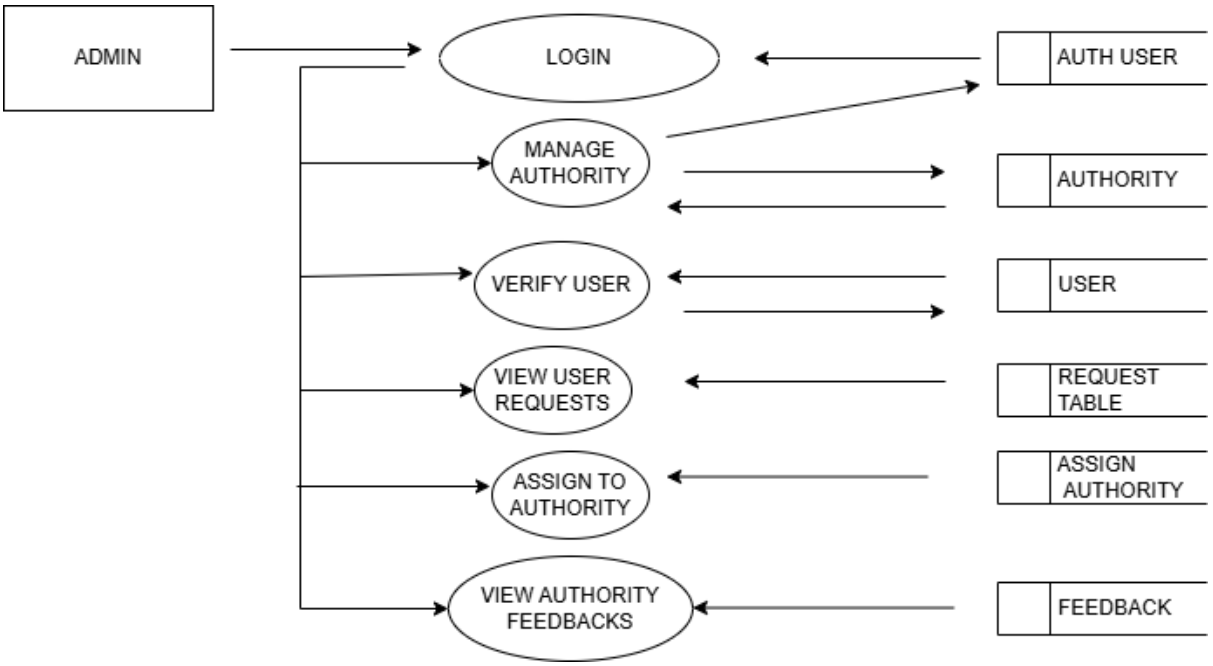
2.3 Information Flow



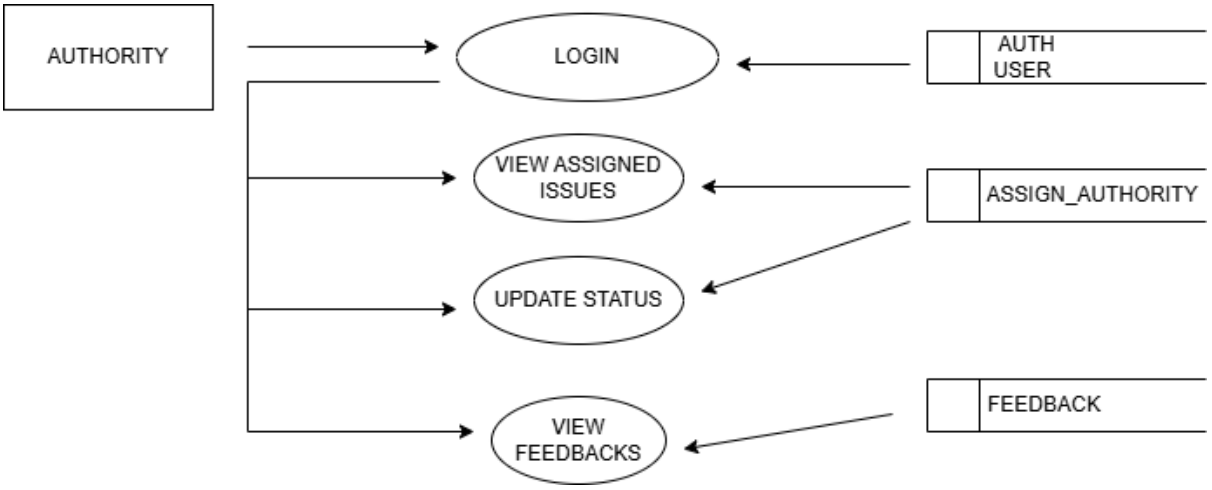
Level 0



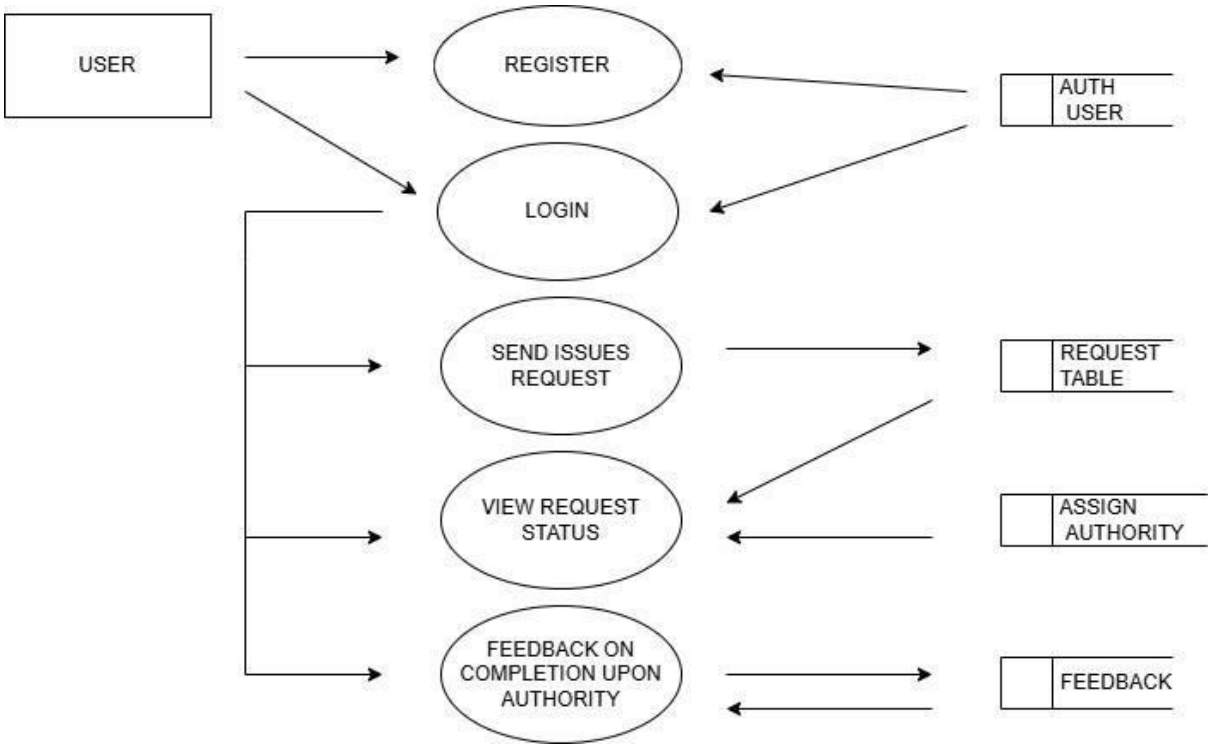
Level 1



Level 2



Level 3



2.4 Components Design

User Module

Handles user registration, login, and profile management. Ensures authentication and validation of user data.

Request Module

Manages complaint submission and tracking. Stores complaint details and status.

Authority Module

Allows authorities to view assigned complaints and update their status.

Admin Module

Responsible for assigning complaints, verifying users, and monitoring system activities.

Backup Module

Handles:

- Database backup
- Media backup
- Compression and storage
- Cloud upload

Restore Module

Responsible for restoring system data from local or cloud backups.

2.5 Key Design Considerations

The system design is based on important factors to ensure performance, reliability, and ease of maintenance.

Scalability

The system is designed to handle increasing users and data efficiently. It can be scaled by improving server resources or integrating with cloud platforms like AWS S3 or Google Cloud Storage. The modular structure allows adding new features without affecting existing components.

Reliability

Reliability is achieved by storing data in multiple locations (local and cloud). Regular automated backups ensure that data is not lost and the system continues to function smoothly even in case of minor failures.

Fault Tolerance

The system includes failover mechanisms to handle failures. If local storage fails, data can be recovered from cloud backups. This ensures continuous system operation with minimal downtime.

Security

The system implements authentication and role-based authorization to control access. User data is stored securely, and cloud access is protected using secure APIs and authentication methods.

Maintainability

The system follows a modular design, making it easy to update, debug, and extend. Clear separation between layers (UI, logic, database) helps in maintaining the system efficiently.

2.6 API Catalogue

The system exposes several APIs for internal and external interaction.

- `/login` - Authenticates user
- `/register` - Registers new user
- `/send-request` - Submit complaint
- `/assign-authority` - Assign complaint

- `/update-status` - Update status
- `/backup` - Trigger backup
- `/restore` - Restore system

---

3. Data Design

3.1 Data Model

The system follows a **relational data model** to organize and manage data efficiently.

Entities

- **User** – Stores user details such as name, email, login credentials, and profile information.
- **Authority** – Contains details of authorities responsible for handling complaints.
- **Request** – Stores complaint information including description, status, and timestamps.
- **Assignment** – Links requests with authorities and tracks assignment details.
- **Feedback** – Stores user feedback after the complaint is resolved.

Relationships

- One **User** can create multiple **Requests** (One-to-Many).
- Each **Request** is assigned to an **Authority** through the **Assignment** module.
- **Feedback** is associated with completed **Requests** to capture user satisfaction.

This structure ensures proper data organization and easy retrieval.

3.2 Data Access Mechanism

The system uses **Django ORM (Object Relational Mapping)** for interacting with the database.

Features

- **Simplifies SQL Queries** – Developers can perform operations without writing complex SQL.
- **Abstraction Layer** – Provides a high-level interface between application and database.
- **Secure Data Handling** – Reduces risk of SQL injection and ensures safe queries.

It supports standard CRUD operations like create, retrieve, update, and delete efficiently.

3.3 Data Retention Policies

To manage storage efficiently and avoid unnecessary data accumulation:

- Only the **last 3 database backups** are retained
- Only the **last 2 media backups** are stored
- Older backups are **automatically deleted**

This ensures optimal use of storage while maintaining recent recovery points.

3.4 Data Migration

Data migration is handled using **Django migration tools**.

- Used to create and update database schema
- Ensures consistency during system updates
- Backup files are used for **data recovery and restoration**
- Supports smooth transition during upgrades without data loss

---

4. Interfaces

User Interface

Web-based interface accessible via browser.

External Interfaces

- Google Drive API (cloud storage)
- MySQL database

---

5. State and Session Management

The system uses Django session management to maintain user state.

- Tracks logged-in users
- Implements role-based access
- Stores session securely

---

6. Caching

Currently not implemented but can be added for performance improvement.

Possible tools:

- Redis
- Django Cache Framework

---

7. Non-Functional Requirements

Non-functional requirements define how the system performs in terms of security, efficiency, and reliability.

7.1 Security Aspects

The system ensures data protection and controlled access through:

- **Authentication and Authorization**
  Users must log in with valid credentials, and access is granted based on permissions.
- **Role-Based Access Control**
  Different roles (User, Admin, Authority) have specific access rights to system features.
- **Secure Password Storage**
  User passwords are stored in encrypted/hashed form to prevent misuse.
- **OAuth Integration for Cloud**
  Secure authentication is used while accessing cloud services like Google Drive.

7.2 Performance Aspects

The system is optimized to provide efficient and smooth operation:

- **Asynchronous Backup Using Threads**
  Backup processes run in the background without affecting system performance.
- **Scheduled Jobs Using APScheduler**
  Backups are automatically triggered at fixed intervals.
- **Efficient File Handling**
  Data and media files are processed and stored efficiently to reduce delays.
- **Optimized Storage Usage**
  Old backups are deleted regularly to manage storage space effectively.

8. References

- Python Documentation
- Django Documentation
- MySQL Documentation
- Google Drive API (PyDrive2)
- Cloud Computing Concepts